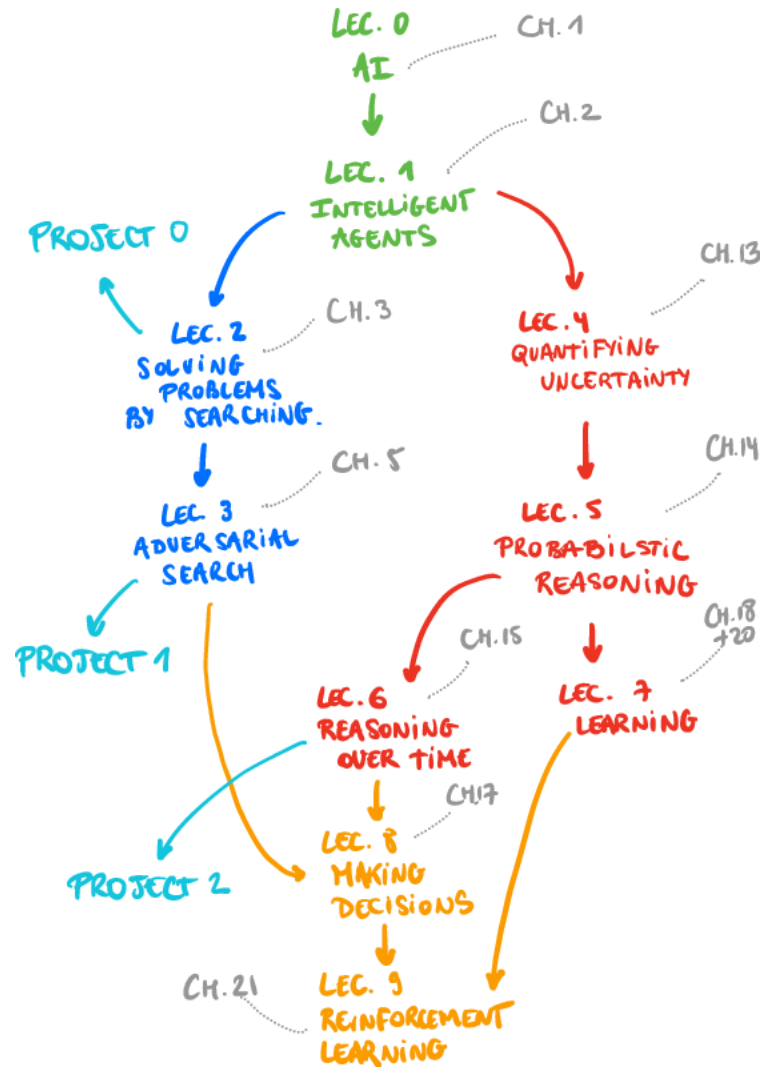


Introduction to Artificial Intelligence

Lecture 8: Making decisions

Prof. Gilles Louppe

g.louppe@uliege.be



Today



Reasoning under uncertainty and **taking decisions**:

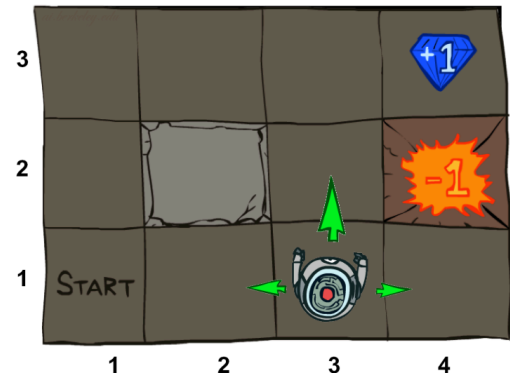
- Markov decision processes
 - MDPs
 - Bellman equation
 - Value iteration
 - Policy iteration
- Partially observable Markov decision processes

Grid world

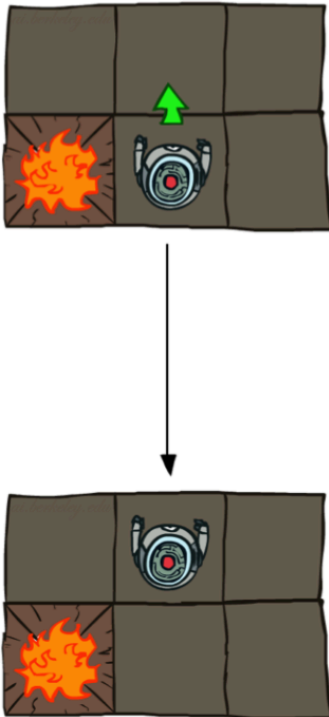
Assume our agent lives in a 3×4 grid environment.

- Noisy movements: actions do not always go as planned.
 - Each action achieves the intended effect with probability **0.8**.
 - The rest of the time, with probability **0.2**, the action moves the agent at right angles to the intended direction.
 - If there is a wall in the direction the agent would have been taken, the agent stays put.
- The agent receives rewards at each time step.
 - Small 'living' reward each step (can be negative).
 - Big rewards come at the end (good or bad).

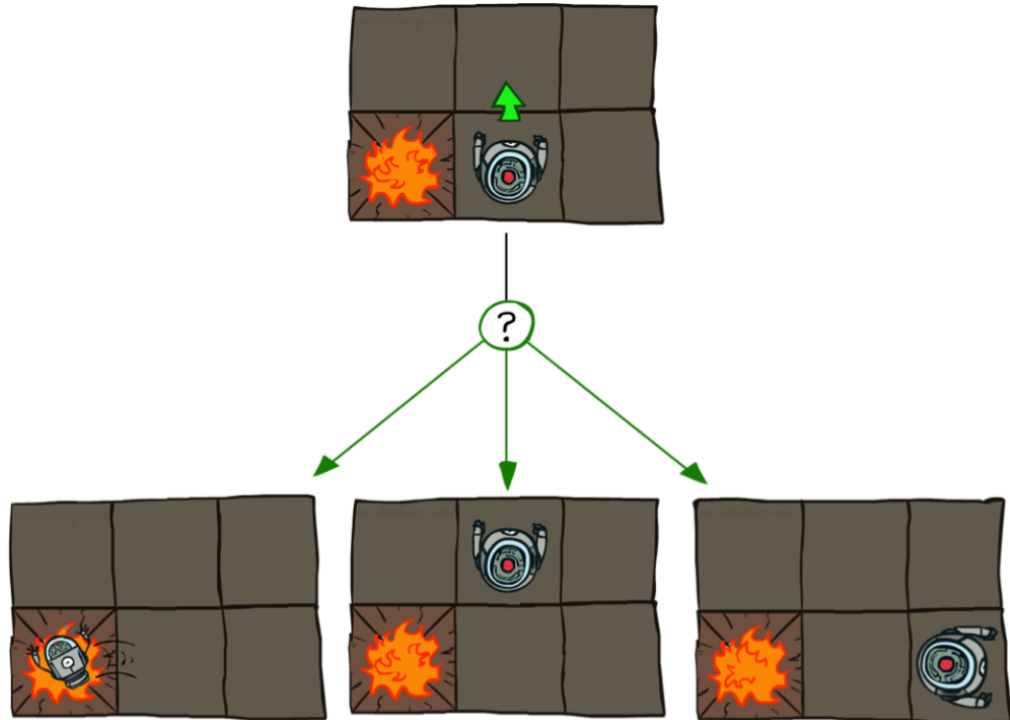
Goal: maximize sum of rewards.



Deterministic actions



Stochastic actions



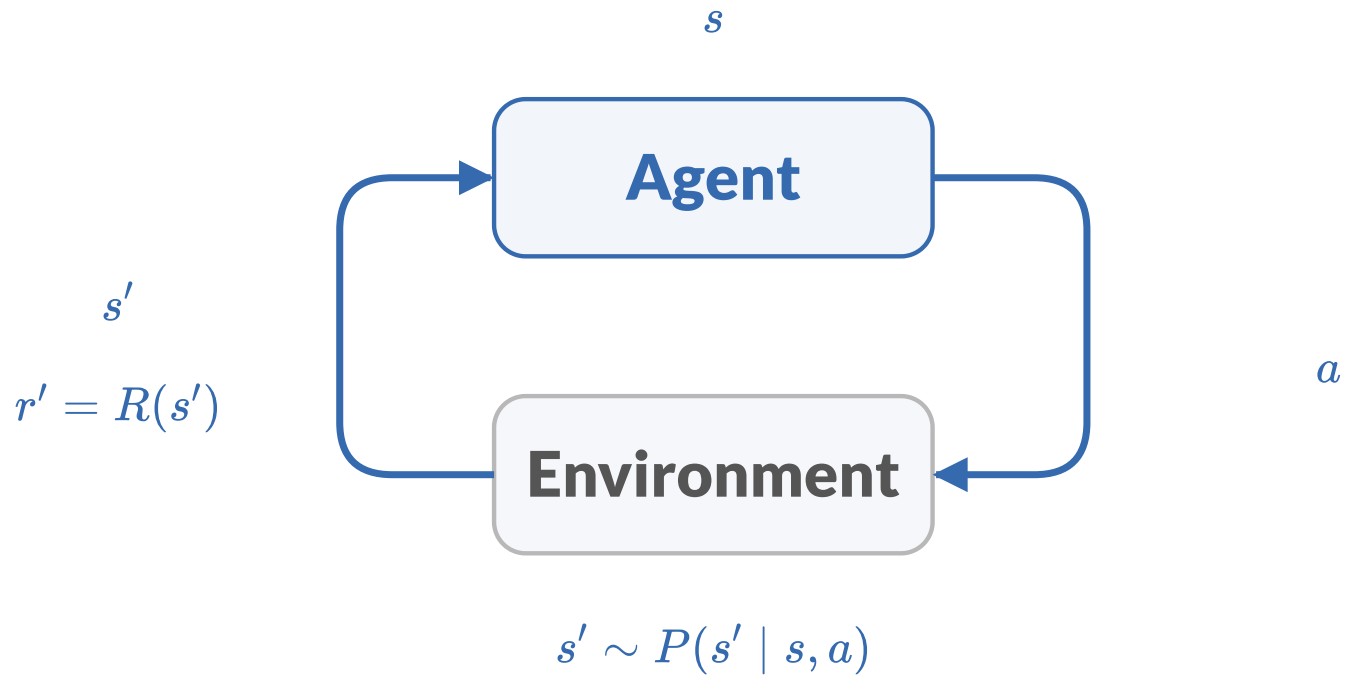
Markov decision processes

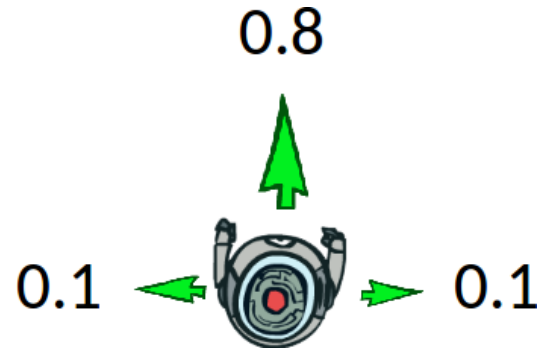
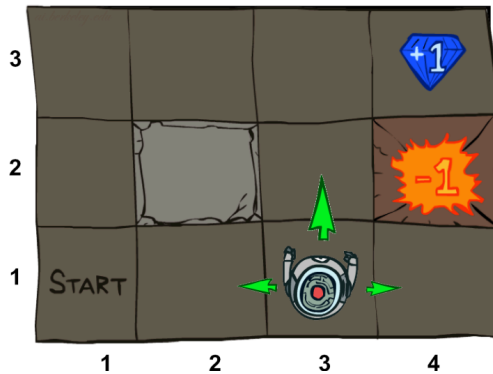
Markov decision processes

A **Markov decision process** (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ such that:

- $\mathcal{S}$ is a set of states s ;
- $\mathcal{A}$ is a set of actions a ;
- P is a (stationary) transition model such that $P(s' \mid s, a)$ denotes the probability of reaching state s' if action a is done in state s ;
- R is a reward function that maps immediate (finite) reward values $R(s)$ obtained in states s ;
- $\gamma \in (0, 1]$ is the discount factor, with $\gamma < 1$ unless every run reaches a terminal state.

Rewards are attached to states. Attaching them to transitions instead, $R(s, a, s')$, gives the same algorithms.





Example

- $\mathcal{S}$: locations (i, j) on the grid.
- $\mathcal{A}$: [Up, Down, Right, Left].
- Transition model: $P(s' \mid s, a)$
- Reward:

$$R(s) = \begin{cases} -0.04 & \text{for non-terminal states} \\ \pm 1 & \text{for terminal states} \end{cases}$$

What is Markovian about MDPs?

Given the present state, the future and the past are independent:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} \mid s_t, a_t)$$

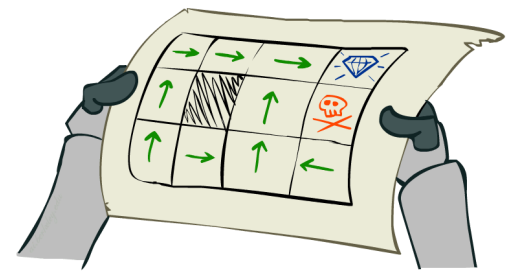
This is similar to search problems, where the successor function could only depend on the current state.



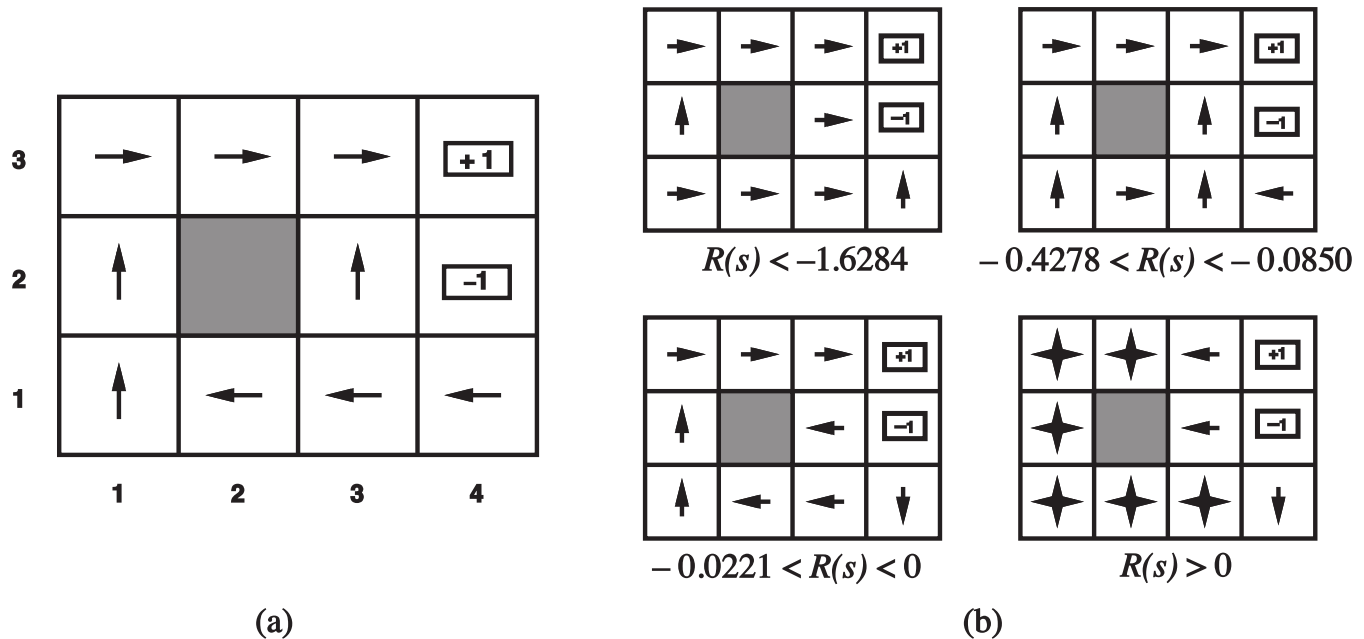
Andrey Markov

Policies

- In deterministic single-agent search problems, our goal was to find an optimal plan, or *sequence* of actions, from start to goal.
- For MDPs, we want to find an optimal **policy** $\pi^* : \mathcal{S} \rightarrow \mathcal{A}$.
 - Lecture 1 writes $\pi : \mathcal{P}^* \rightarrow \mathcal{A}$ on percept sequences; here the state is observed and Markovian, so the current state is enough.
 - An optimal policy is one that maximizes the expected utility, e.g. the expected sum of rewards.
 - An explicit policy defines a reflex agent.
- Expectiminimax computed no policy, only an action for a single state.



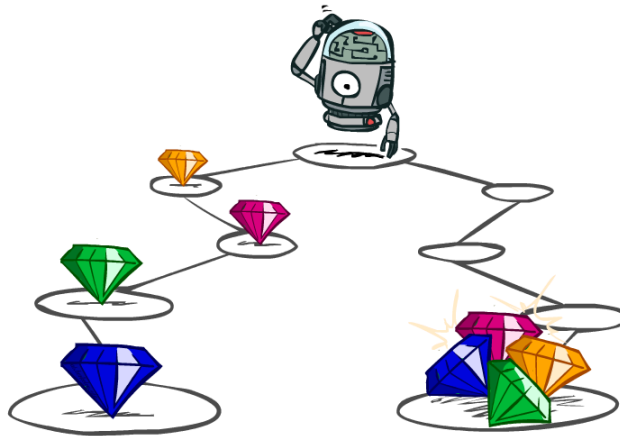
Optimal policy when $R(s) = -0.04$ for all non-terminal states s .



(a) Optimal policy when $R(s) = -0.04$ for all non-terminal states s . (b) Optimal policies for four different ranges of $R(s)$.

Depending on $R(s)$, the **balance between risk and reward** changes from risk-taking to very conservative.

Utilities over time



What preferences should an agent have over state or reward sequences?

- More or less? $[2, 3, 4]$ or $[1, 2, 2]$?
- Now or later? $[1, 0, 0]$ or $[0, 0, 1]$?

Lecture 1 scores a run with $V(s_{1:T})$. With a reward $R(s_t)$ at each step, this amounts to scoring its reward sequence.

Theorem. If we assume **stationary** preferences over reward sequences, i.e. such that

$$[r_0, r_1, r_2, \dots] \succ [r_0, r'_1, r'_2, \dots] \Rightarrow [r_1, r_2, \dots] \succ [r'_1, r'_2, \dots],$$

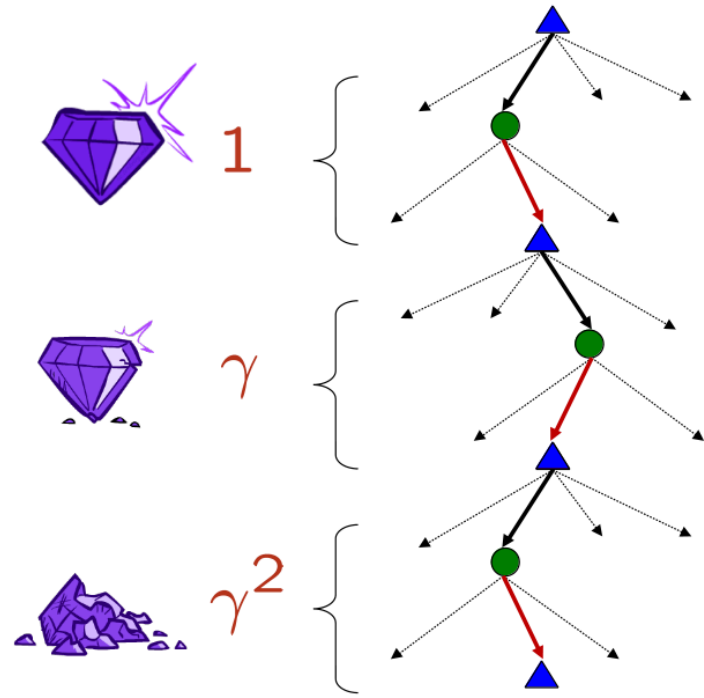
then there are only two coherent ways to assign utilities to sequences:

Additive utility: $V([r_0, r_1, r_2, \dots]) = r_0 + r_1 + r_2 + \dots$

Discounted utility: $V([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$
($0 < \gamma < 1$)

Discounting

- Each time we transition to the next state, we multiply in the discount once.
- Why discount?
 - Sooner rewards probably do have higher utility than later rewards.
 - Will help our algorithms converge.



Example: discount $\gamma = 0.5$

- $V([1, 2, 3]) = 1 + 0.5 \times 2 + 0.25 \times 3$
- $V([1, 2, 3]) < V([3, 2, 1])$

Infinite sequences

What if the agent lives forever? Do we get infinite rewards? Comparing reward sequences with $+\infty$ utility is problematic.

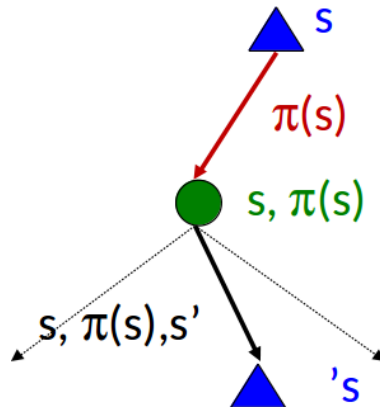
Solutions:

- Finite horizon: (similar to depth-limited search)
 - Terminate episodes after a fixed number of steps T .
 - Results in non-stationary policies (π depends on time left).
- Discounting (with $0 < \gamma < 1$ and rewards bounded by $\pm R_{\max}$):

$$V([r_0, r_1, \dots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq \frac{R_{\max}}{1 - \gamma}$$

Smaller γ results in a shorter horizon.

- Absorbing state: guarantee that for every policy, a terminal state will eventually be reached.



Policy evaluation

The **expectation** of $f(X)$ is $\mathbb{E}[f(X)] = \sum_x P(x)f(x)$. The expected utility of executing π from s is therefore

$$V^\pi(s) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t R(s_t) \mid s_0 = s, \pi \right],$$

over the state sequences that π generates from s .

Optimal policies

Among all policies the agent could execute, the **optimal policy** is the policy π_s^* that maximizes the expected utility:

$$\pi_s^* = \arg \max_{\pi} V^{\pi}(s)$$

Because of discounted utilities, the optimal policy is *independent* of the starting state s . Therefore we simply write π^* .

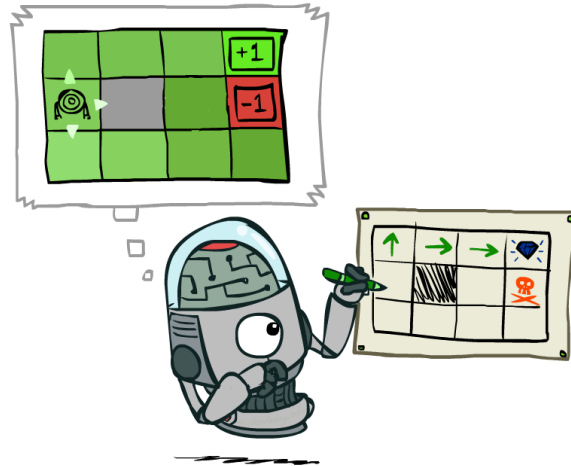
Values of states

The utility, or value, $V(s)$ of a state is now simply defined as $V^{\pi^*}(s)$.

- That is, the expected (discounted) reward if the agent executes an optimal policy starting from s .
- Notice that $R(s)$ and $V(s)$ are quite different quantities:
 - $R(s)$ is the short term reward for having reached s .
 - $V(s)$ is the long term total reward from s onward.

3	0.812	0.868	0.918	+ 1
2	0.762		0.660	-1
1	0.705	0.655	0.611	0.388
	1	2	3	4

Utilities of the states in Grid World, calculated with $\gamma = 1$ and $R(s) = -0.04$ for non-terminal states.



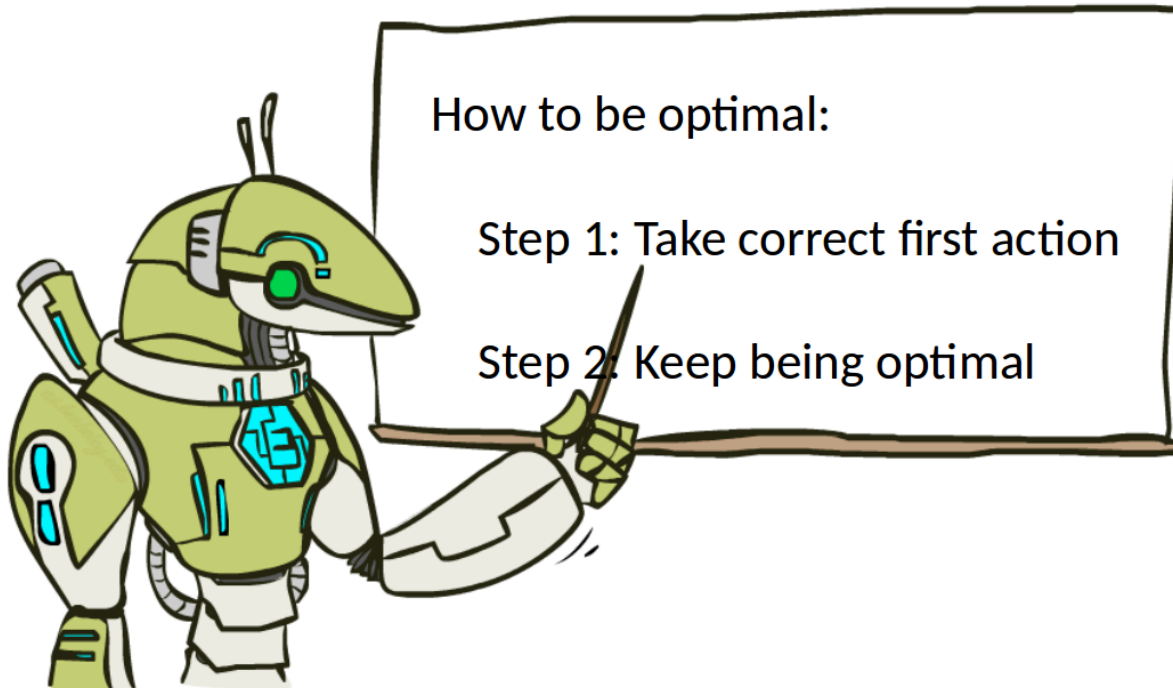
Policy extraction

Using the principle of maximum expected utility, the optimal action maximizes the expected utility of the subsequent state. That is,

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s' \mid s, a) V(s').$$

Therefore, we can extract the optimal policy provided we can estimate the utilities of states.

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s' | s, a) V(s')$$



The Bellman equation

The utility of a state is the immediate reward for that state, plus the expected discounted utility of the next state, assuming that the agent chooses the optimal action:

$$V(s) = R(s) + \gamma \max_a \sum_{s'} P(s' | s, a) V(s').$$

- These equations are called the **Bellman equations**. They form a system of $n = |\mathcal{S}|$ non-linear equations with as many unknowns.
- The utilities of states, defined as the expected utility of subsequent state sequences, are solutions of the set of Bellman equations.

Example

$$V(1, 1) = -0.04 + \gamma \max[0.8V(1, 2) + 0.1V(2, 1) + 0.1V(1, 1), \\ 0.9V(1, 1) + 0.1V(1, 2), \\ 0.9V(1, 1) + 0.1V(2, 1), \\ 0.8V(2, 1) + 0.1V(1, 2) + 0.1V(1, 1)]$$

Value iteration

Because of the **max** operator, the Bellman equations are non-linear and solving the system is problematic.

The **value iteration** algorithm provides a fixed-point iteration procedure for computing the state utilities $V(s)$:

- Let $V_i(s)$ be the estimated utility value for s at the i -th iteration step.
- The **Bellman update** consists in updating simultaneously all the estimates to make them *locally consistent* with the Bellman equation:

$$V_{i+1}(s) = R(s) + \gamma \max_a \sum_{s'} P(s' | s, a) V_i(s')$$

- Repeat until convergence.

```

function VALUE-ITERATION( $mdp, \varepsilon$ ) returns the values  $V$ 
   $V_0(s) \leftarrow 0$ , for each state  $s$  of  $S$ 
  for  $i = 0, 1, 2, \dots$  do
    for each state  $s$  of  $S$  do
       $V_{i+1}(s) \leftarrow R(s) + \gamma \max_a \sum_{s'} P(s' | s, a) V_i(s')$ 
    if  $\max_s |V_{i+1}(s) - V_i(s)| < \varepsilon(1 - \gamma)/\gamma$  then return  $V_{i+1}$ 

```

(Step-by-step code example)

`demo/lec8/mdp.ipynb`

Convergence

Let V_i and V_{i+1} be successive approximations to the true utility V .

Theorem. For any two approximations V_i and V'_i ,

$$\|V_{i+1} - V'_{i+1}\|_{\infty} \leq \gamma \|V_i - V'_i\|_{\infty}.$$

- That is, the Bellman update is a contraction by a factor γ on the space of utility vectors.
- Therefore, any two approximations must get closer to each other, and in particular any approximation must get closer to the true V .

$\Rightarrow$ Value iteration always converges to a unique solution of the Bellman equations whenever $\gamma < 1$.

Proof.

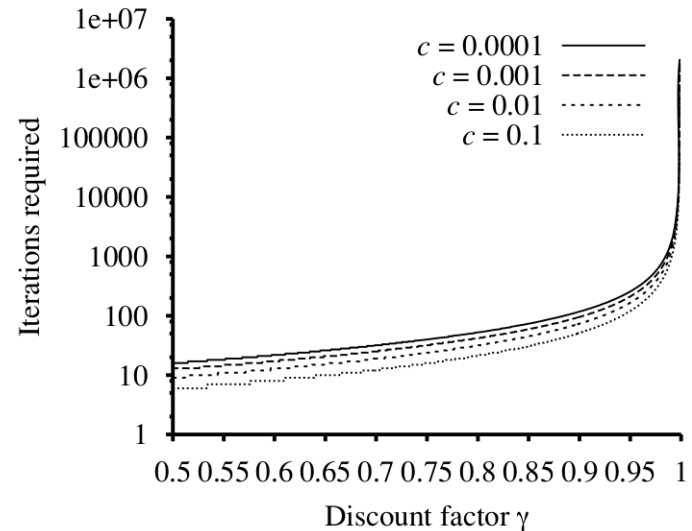
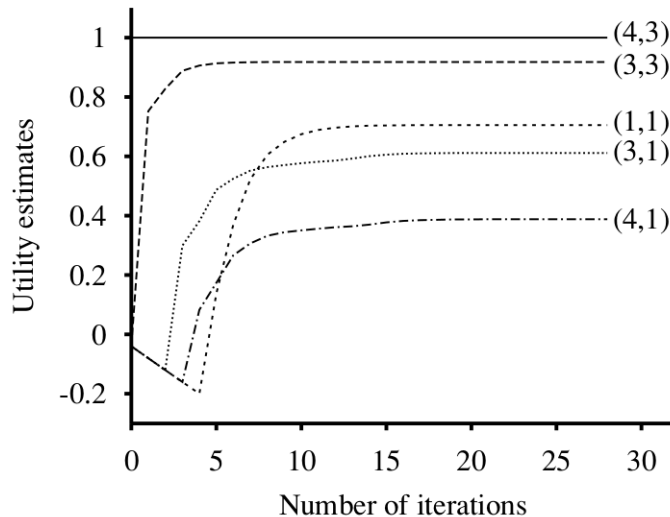
$$\begin{aligned}
& \|V_{i+1} - V'_{i+1}\|_\infty \\
&= \max_s |V_{i+1}(s) - V'_{i+1}(s)| \\
&= \max_s \left| R(s) + \gamma \max_a \sum_{s'} P(s' | s, a) V_i(s') - R(s) - \gamma \max_a \sum_{s'} P(s' | s, a) V'_i(s') \right| \\
&= \gamma \max_s \left| \max_a \sum_{s'} P(s' | s, a) V_i(s') - \max_a \sum_{s'} P(s' | s, a) V'_i(s') \right| \\
&\leq \gamma \max_s \max_a \left| \sum_{s'} P(s' | s, a) V_i(s') - \sum_{s'} P(s' | s, a) V'_i(s') \right| \\
&= \gamma \max_s \max_a \left| \sum_{s'} P(s' | s, a) (V_i(s') - V'_i(s')) \right| \\
&\leq \gamma \max_s \max_a \sum_{s'} P(s' | s, a) |V_i(s') - V'_i(s')| \\
&\leq \gamma \max_s \max_a \sum_{s'} P(s' | s, a) \|V_i - V'_i\|_\infty \\
&= \gamma \|V_i - V'_i\|_\infty
\end{aligned}$$

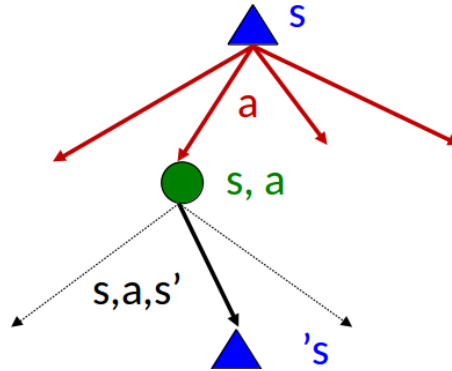
Performance

Since $\|V_{i+1} - V\|_\infty \leq \gamma \|V_i - V\|_\infty$, the error is reduced by a factor of at least γ at each iteration.

Therefore, value iteration converges exponentially fast:

- The maximum initial error is $\|V_0 - V\|_\infty \leq 2R_{\max}/(1 - \gamma)$.
- To reach an error of at most ϵ after N iterations, we require $\gamma^N 2R_{\max}/(1 - \gamma) \leq \epsilon$.





Problems with value iteration

Value iteration repeats the Bellman updates:

$$V_{i+1}(s) = R(s) + \gamma \max_a \sum_{s'} P(s' \mid s, a) V_i(s')$$

- Problem 1: it is slow – $O(|\mathcal{S}|^2|\mathcal{A}|)$ per iteration.
- Problem 2: the **max** at each state rarely changes.
- Problem 3: the policy π_i extracted from the estimate V_i might be optimal even if V_i is inaccurate!

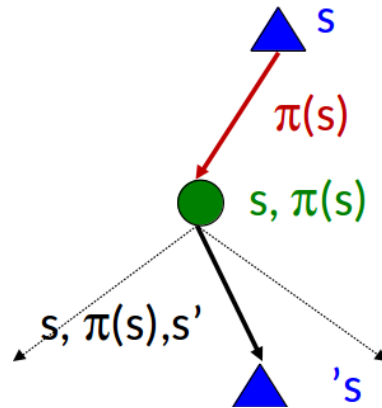
Policy iteration

The **policy iteration** algorithm instead directly computes the policy (instead of state values). It alternates the following two steps:

- Policy evaluation: given π_i , calculate $V_i = V^{\pi_i}$, i.e. the utility of each state if π_i is executed.
- Policy improvement: calculate a new policy π_{i+1} using one-step look-ahead based on V_i :

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} P(s' | s, a) V_i(s')$$

This algorithm is still optimal, and might converge (much) faster under some conditions.



Policy evaluation

At the i -th iteration we have a simplified version of the Bellman equations that relate the utility of s to the utilities of its neighbors:

$$V_i(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) V_i(s')$$

These equations are now **linear** because the **max** operator has been removed.

- for n states, we have n equations with n unknowns;
- this can be solved exactly in $O(n^3)$ by standard linear algebra methods.

In some cases $O(n^3)$ is too prohibitive. Fortunately, it is not necessary to perform exact policy evaluation. An approximate solution is sufficient.

One way is to run k iterations of simplified Bellman updates:

$$V_{i+1}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) V_i(s')$$

This hybrid algorithm is called **modified policy iteration**.

```

function POLICY-ITERATION(mdp) returns a policy  $\pi$ 
   $\pi_0(s) \leftarrow$  an action, for each state  $s$  of  $S$ 
  for  $i = 0, 1, 2, \dots$  do
     $V_i \leftarrow$  POLICY-EVALUATION( $\pi_i, mdp$ )
     $\pi_{i+1}(s) \leftarrow \operatorname{argmax}_a \sum_{s'} P(s' | s, a) V_i(s'),$  for each state  $s$  of  $S$ 
    if  $\pi_{i+1} = \pi_i$  then return  $\pi_i$ 

```

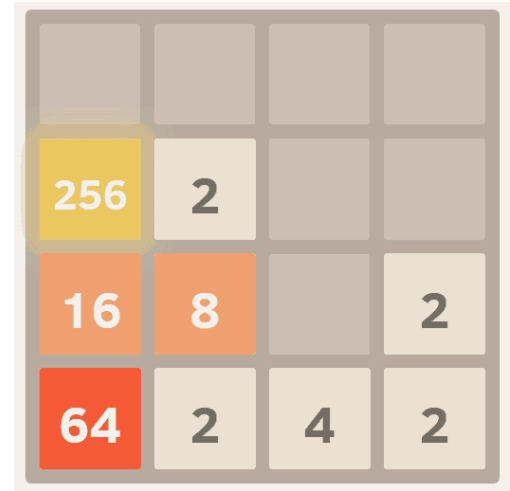
(Step-by-step code example)

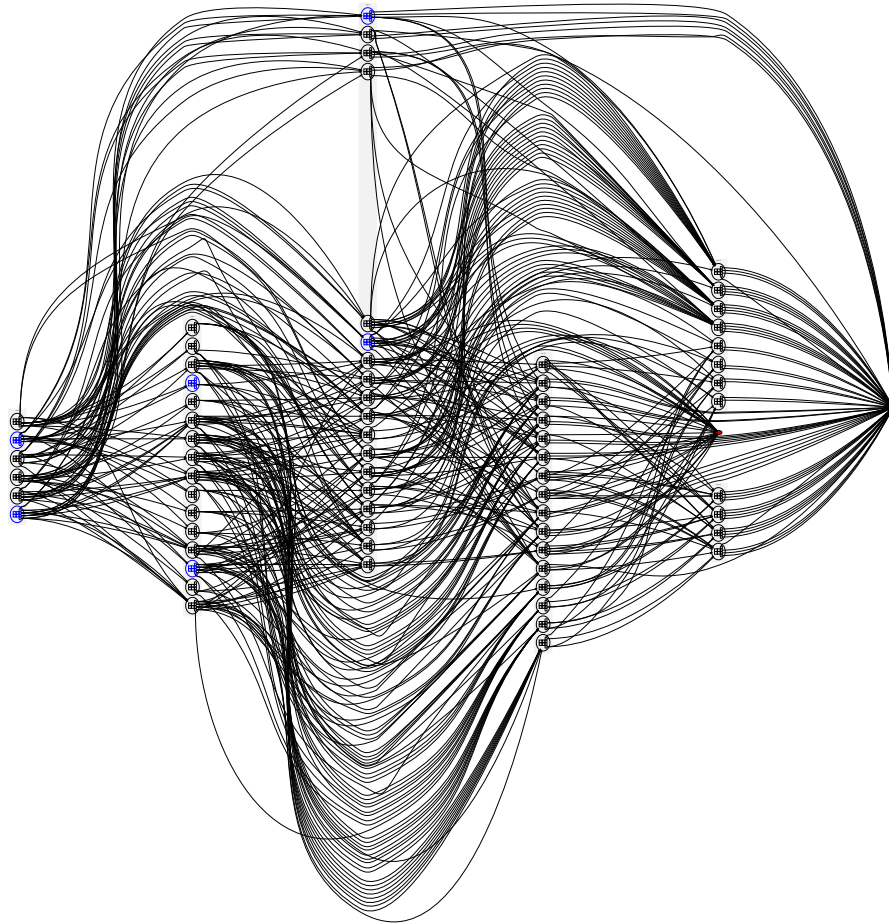
`demo/lec8/mdp.ipynb`

Recap example: 2048

The game 2048 is a Markov decision process!

- $\mathcal{S}$: all possible configurations of the board (huge!)
- $\mathcal{A}$: swiping left, right, up or down.
- $P(s' \mid s, a)$: encodes the game's dynamic
 - collapse matching tiles
 - place a random tile on the board
- $R(s) = 1$ if s is a winning state, and 0 otherwise.





The transition model for a 2×2 board and a winning state at 8.

Optimal play for a 3×3 grid and a winning state at 1024.

See jdlm.info: The Mathematics of 2048.

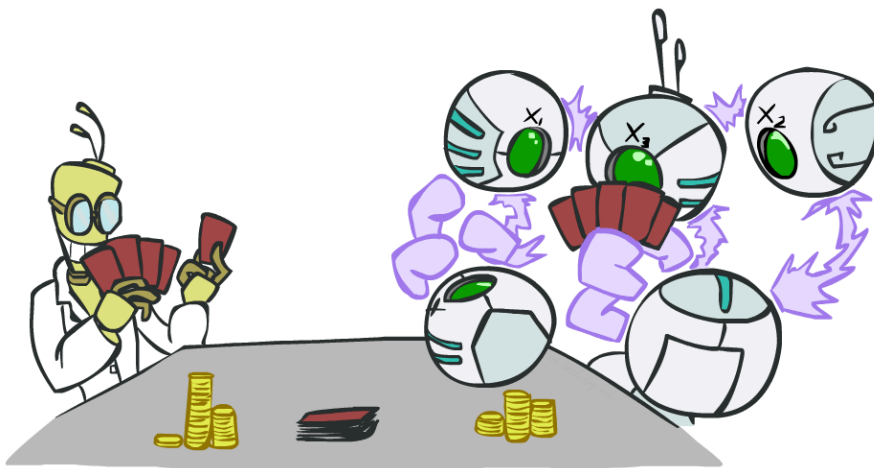
Partially observable Markov decision processes

POMDPs

What if the environment is only **partially observable**?

- The agent does not know in which state s it is in.
 - Therefore, it cannot evaluate the reward $R(s)$ associated to the unknown state.
 - Also, it makes no sense to talk about a policy $\pi(s)$.
- Instead, the agent collects percepts $e \in \mathcal{E}$ (Lecture 1) through a sensor model $P(e \mid s)$, from which it can reason about the unknown state s .

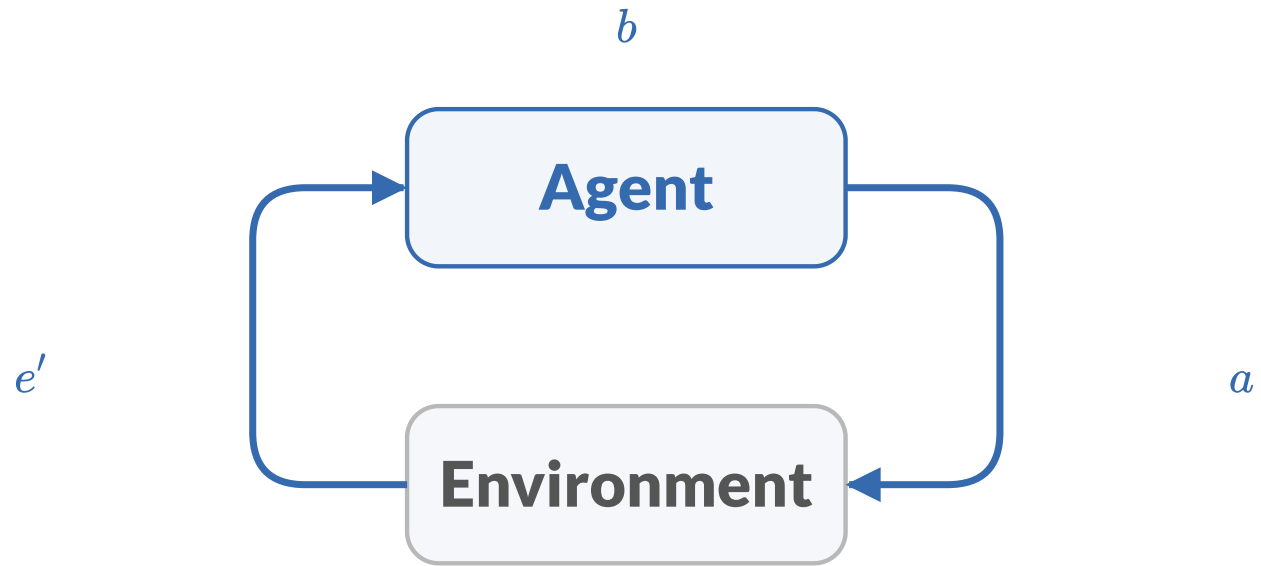
A **POMDP** is the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma, \mathcal{E}, P(e \mid s))$.



We will assume that the agent maintains a belief state b .

- b represents a probability distribution $\mathbf{P}(S)$ of the current agent's beliefs over its state;
- $b(s)$ denotes the probability $P(S = s)$ under the current belief state;
- the belief state b is updated as evidence e are collected.

This is filtering (Lecture 6), with the action a in the transition model.



$$s' \sim P(s' \mid s, a)$$

$$e' \sim P(e' \mid s')$$

Belief MDP

Theorem (Åström, 1965). The optimal action depends only on the agent's current belief state.

- The optimal policy can be described by a mapping $\pi^*(b)$ from beliefs to actions.
- It does not depend on the actual state the agent is in.

In other words, POMDPs can be reduced to an MDP in belief-state space, provided we can define a transition model $P(b' \mid b, a)$ and a reward function ρ over belief states.

If b was the previous belief state and the agent does action a and perceives e , then the new belief state over S' is given by

$$b' = \alpha \mathbf{P}(e \mid S') \sum_s \mathbf{P}(S' \mid s, a) b(s) = \alpha \text{forward}(b, a, e).$$

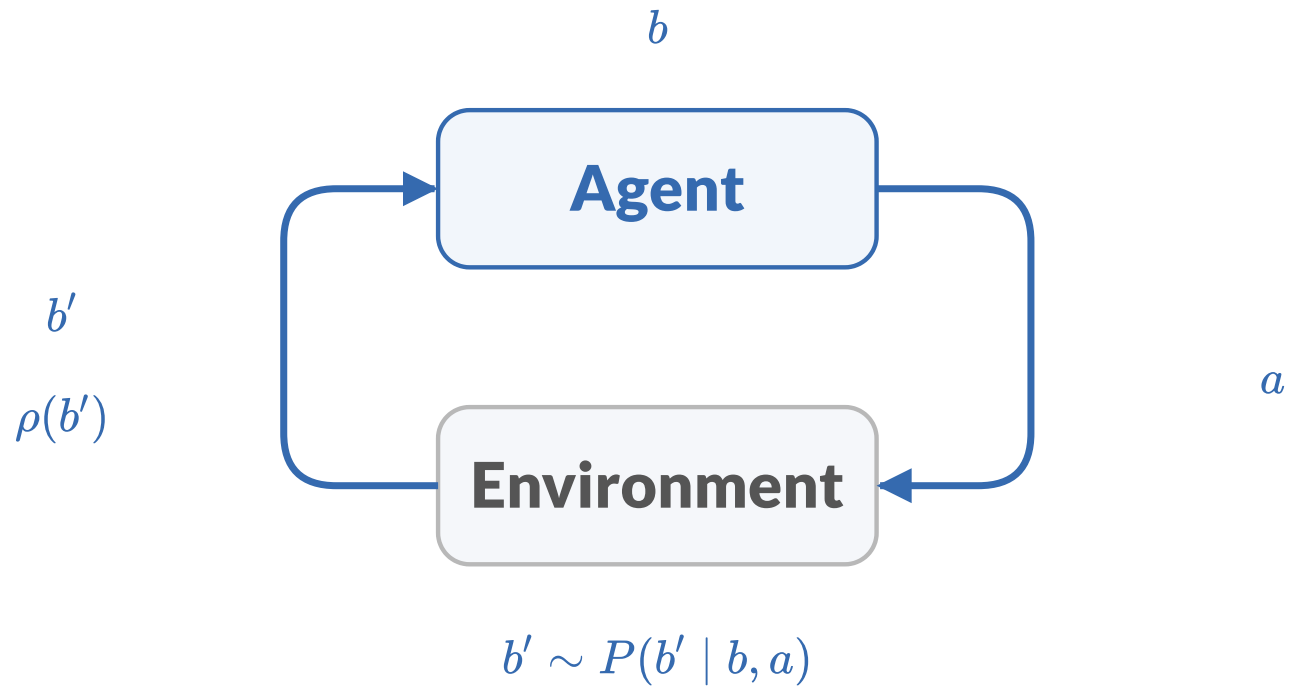
Therefore,

$$\begin{aligned} P(b' \mid b, a) &= \sum_e P(b', e \mid b, a) \\ &= \sum_e P(b' \mid b, a, e) P(e \mid b, a) \\ &= \sum_e P(b' \mid b, a, e) \sum_{s'} P(e \mid b, a, s') P(s' \mid b, a) \\ &= \sum_e P(b' \mid b, a, e) \sum_{s'} P(e \mid s') \sum_s P(s' \mid s, a) b(s) \end{aligned}$$

where $P(b' \mid b, a, e) = 1$ if $b' = \text{forward}(b, a, e)$ and 0 otherwise.

We can also define a reward function for belief states as the expected reward for the actual state the agent might be in:

$$\rho(b) = \sum_s b(s)R(s)$$



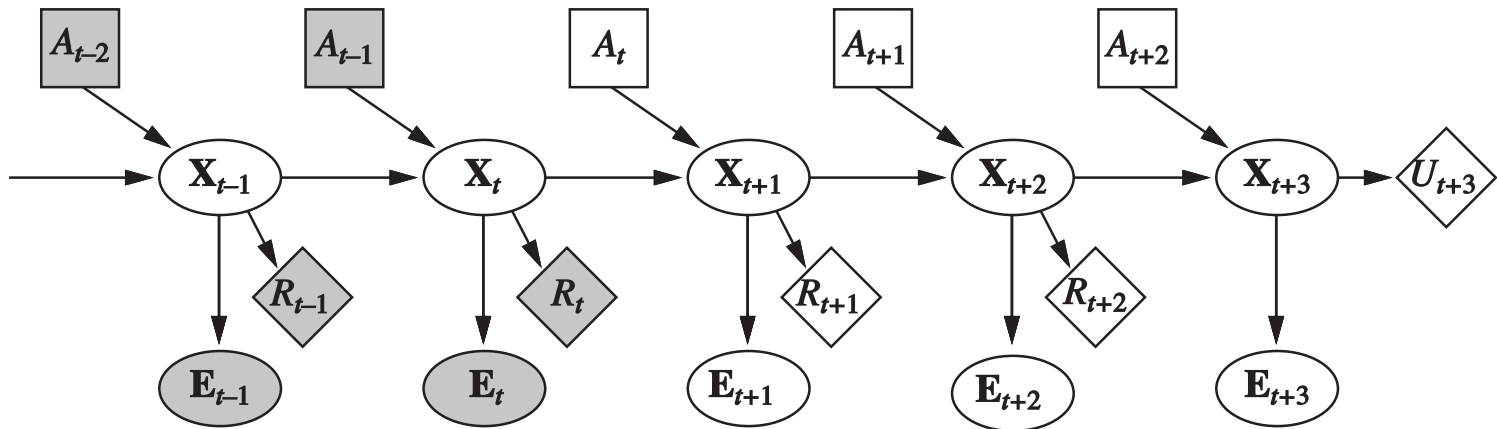
Although we have reduced POMDPs to MDPs, the Belief MDP we obtain has a **continuous** (and usually high-dimensional) state space.

- None of the algorithms described earlier directly apply.
- In fact, solving POMDPs remains a difficult problem for which there is no known efficient exact algorithm.
- Yet, most real-world decision making problems are partially observable.

Online agents

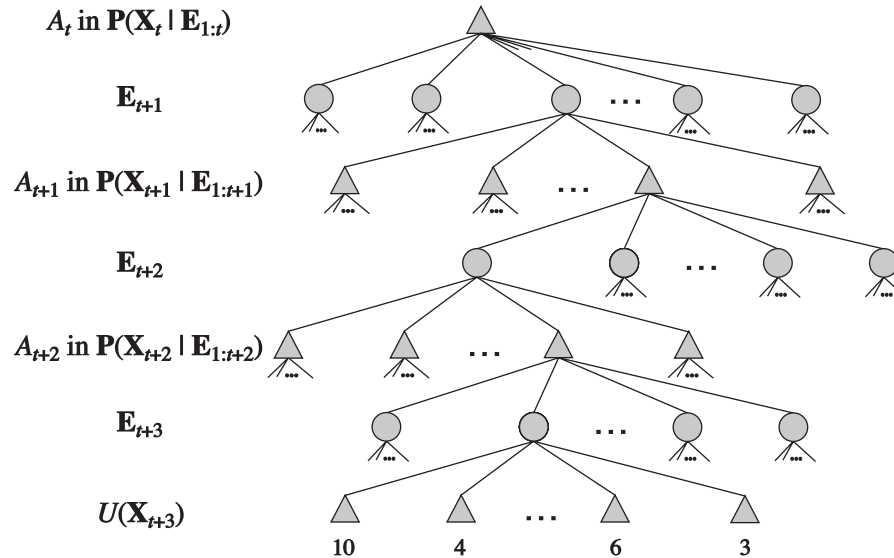
While it is difficult to directly derive π^* , a decision-theoretic agent can be constructed for POMDPs:

- The transition and sensor models are represented by a *dynamic Bayesian network*;
- The dynamic Bayesian network is extended with decision (A) and utility (R and U) nodes to form a dynamic decision network;
- A **filtering algorithm** is used to incorporate each new percept and action and to update the belief state representation;
- Decisions are made by projecting forward possible action sequences and choosing (approximately) the best one, in a manner similar to a truncated expectiminimax.



At time t , the agent must decide what to do.

- Shaded nodes represent variables with known values.
- The network is unrolled for a finite horizon.
- It includes reward nodes for X_{t+1} and X_{t+2} , and a node U_{t+3} for the estimated value V of X_{t+3} .



Part of the look-ahead solution of the previous decision network:

- Each triangular node is a belief state in which the agent makes a decision.
 - The belief state at each node can be computed by applying a filtering algorithm to the sequence of percepts and actions leading to it.
- The round nodes correspond to choices by the environment.

A decision can be extracted from the search tree by backing up the (estimated) utility values from the leaves, taking the average at the chance nodes and taking the maximum at the decision nodes.

Summary

- An MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ scores a run by its discounted rewards $\sum_{t \geq 0} \gamma^t R(s_t)$. Its solution is a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$, optimal when it maximizes $V^\pi(s)$ in every state.
- The values of the optimal policy satisfy the Bellman equation,

$$V(s) = R(s) + \gamma \max_a \sum_{s'} P(s' | s, a) V(s'),$$

from which $\pi^*(s) = \arg \max_a \sum_{s'} P(s' | s, a) V(s')$.

- Value iteration solves it by iterating the Bellman update, a contraction of factor γ ; policy iteration alternates evaluation and improvement, and stops when the policy no longer changes.
- A POMDP is an MDP over belief states, where the update is filtering. Exact solutions are out of reach, but an agent can still look ahead with a decision network.

